

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)

Assessment Tool Weightage: 30%

Dr Dumpala Prasanth

QUESTIONS: 10

Total Marks:50

Annexure A

Constraint - Based Problem-Solving

Duration: 2Hrs

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.
(10 Marks)
2. Implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ϵ values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.
(10 Marks)
3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.
(10 Marks)
4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.
(10 Marks)
5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of

deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

(10 Marks)

6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.

(10 Marks)

7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization.

(10 Marks)

8. Analyse an experimental comparison between random action selection and ϵ -greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

(10 Marks)

9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process.

(10 Marks)

10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

(10 Marks)

Code of Conduct:

I N. Lokesh (1924253²⁹) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4	Level 3	Level 2	Level 1
----------	-----------	---------	---------	---------	---------

	*	(Exemplary)	(Proficient)	(Developing)	(Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

2. Reinforcement learning in OpenAI GYM for Grid world navigation.

Explanation

- A Grid world consists of cells represents a state where an agent starts from an initial position and must reach the goal while avoiding obstacles.

- State space: Each grid cell represents a state.

- Action space: UP, DOWN, left, Right.

- Reward function:

 - Goal: +10

 - obstacle: -10

 - Normal move: -1

- Constraints:

 - cannot move outside the grid.

 - obstacles cannot be crossed

Python code:-

```
import gym
```

```
import numpy as np
```

```
rows, cols = 5, 5
```

goal = (4, 4)

obstacles = [(2, 2), (3, 1)]

state = (0, 0)

actions = ["up", "Down", "LEFT", "RIGHT"]

Print("Start:", state)

Print("Goal:", goal)

Print("Available Actions:", actions)

2) ϵ - Greedy Multi-Armed Bandit Algorithm.

Explanation:-

The ϵ -greedy algorithm balances exploration and exploitation.

- $\epsilon = 0.1 \rightarrow$ Mostly Exploitation

- $\epsilon = 0.3 \rightarrow$ Balanced

- $\epsilon = 0.5 \rightarrow$ More exploration.

Budget = 500 trials.

Python code

```
import numpy as np
```

```
arms = [0.2, 0.5, 0.8]
```

```
epsilon = 0.3
```

$Q = np.zeros(3)$

$N = np.zeros(3)$

for i in range(500):

if np.random.rand() < epsilon:

action = np.random.randint(3)

else:

action = np.argmax(Q)

reward = np.random.rand() < costs[action]

$N[action] += 1$

$Q[action] += (reward - Q[action]) / N[action]$

print(Q)

Comparison

ϵ	Exploration	Exploitation
0.1	Low	High
0.3	Medium	Medium
0.5	High	Low

Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption.

Answer)

A Markov Decision Process (MDP) is a mathematical model used in Reinforcement Learning to make optimal decisions in a sequential environment. An autonomous robot uses an MDP to find the shortest and most energy-efficient path to its destination.

MDP Components

States

- Robot locations.

Actions

- Move up
- Move Down
- Move left
- Move Right

Transition Probability

- Correct move = 0.9
- Wrong move = 0.1

Rewards

- Goal = +100
- Normal step = -1
- Collision = -20

Energy constraint

- Battery capacity limits total moves
- Robot selects shortest path.

value iteration Algorithm using Bellman equation

The Bellman equation updates the value of every state until convergence.

Python code

```
import numpy as np
```

```
gamma = 0.9
```

```
V = np.zeros((4,4))
```

```
for _ in range(20):
```

```
    new V = V.copy()
```

```
    for i in range(4):
```

```
        for j in range(4):
```

```
            new V[i,j] = Max(V[i,j], 1)
```

```
    V = new V
```

```
print(V)
```

Restricted states

- Agent cannot enter blocked cells.
- These states receive very low reward.

5) RL Framework for warehouse Robot.

Constraints

- Battery = 30 minutes
- maximum deliveries before battery ends

Policy

Choose the nearest delivery location

Rewards

Action	Reward
Successful delivery	+20
Battery saving	+5
Late delivery	-10
Battery exhausted	-30

Actions

- Move up
- Move Down
- Move Left
- Move Right
- Pick up package
- Delivery package
- Delivery
- Return to charging station.

Environment

- Warehouse with delivery locations, charging station, and obstacles.

States

- Robot's current location
- Battery level
- Number of deliveries completed.

Conclusion

The RL agent learns an optimal policy that balances delivery efficiency and battery conservation. By using suitable rewards and constraints, the warehouse robot maximizes the Number of deliveries while ensuring it never exceeds its 30 minute battery limit.

✓
✓
✓